

CHAPTER 1 · SUB-CHAPTER 1.6

Collections – Internals, Algorithms & Best Practices

Fail-Fast vs Fail-Safe · Collections Utility Class · Sorting Algorithms
Complexity Tables · Iterator Pattern · Java 9+ Immutable Factories
Anti-Patterns · Best Practices · Collection Selection Guide

~20 Pages

52+ Q&A

Adv. Questions

Interview Ready

■ Purple [ADV] questions indicate advanced / senior-level depth

Section 1 — Fail-Fast vs Fail-Safe Iterators

1.1 Overview

Java collections fall into two categories regarding concurrent modification during iteration: **fail-fast** iterators (most collections) immediately throw **ConcurrentModificationException** when the collection is structurally modified outside the iterator, while **fail-safe** iterators (concurrent collections) either work on a snapshot or use weak-consistency semantics.

1.2 modCount Mechanism (Fail-Fast)

- Every structural modification (add, remove, clear, etc.) increments the **modCount** field in `AbstractList/AbstractMap`
- When an iterator is created, it captures **expectedModCount = modCount**
- Each call to **next()** or **remove()** checks: if (modCount != expectedModCount) throw new `ConcurrentModificationException()`
- **Structural modification**: any operation that changes the size or internal structure (add, remove, clear, `ensureCapacity` triggering `resize`)
- **Non-structural modification**: `set()` — changes a value but not the size or structure; does NOT increment `modCount`
- Collections: `ArrayList`, `LinkedList`, `HashMap`, `HashSet`, `LinkedHashMap`, `TreeMap`, `TreeSet` are all fail-fast

```
// Fail-fast demonstration
List<String> list = new ArrayList<>(Arrays.asList("a","b","c"));
Iterator<String> it = list.iterator();
list.add("d"); // structural modification – increments modCount
it.next(); // throws ConcurrentModificationException!

// Safe removal via iterator.remove()
Iterator<String> it2 = list.iterator();
while (it2.hasNext()) {
    String s = it2.next();
    if (s.equals("b")) it2.remove(); // safe: updates expectedModCount
}

// Safe removal via removeIf (Java 8+)
list.removeIf(s -> s.equals("b")); // internally uses iterator.remove()

// Safe removal via for-each with removal collected, then removeAll
List<String> toRemove = new ArrayList<>();
for (String s : list) { if (s.startsWith("a")) toRemove.add(s); }
list.removeAll(toRemove);
```

1.3 Fail-Safe Collections

Collection	Strategy	Behavior During Iteration
CopyOnWriteArrayList	Full array copy on write	Iterates over snapshot at iterator creation time; never throws CME
CopyOnWriteArraySet	Full array copy on write	Same as CopyOnWriteArrayList — uses it internally
ConcurrentHashMap	Weak-consistency / segment locking	Iterator sees state as of creation or later; may or may not see concurrent changes
ConcurrentSkipListMap	Non-blocking skip list	Weakly consistent; no CME; reflects some concurrent changes
ConcurrentLinkedQueue	Non-blocking linked structure	Weakly consistent; traversal safe but may miss recent additions

1.4 Weak-Consistency Explained

`ConcurrentHashMap`'s iterator captures a reference to the internal **Node[] table** at construction time. It then traverses actual next pointers of nodes. This means: (1) elements present at iterator creation will be seen; (2) elements inserted or removed *after* creation may or may not be reflected — this is the weak-consistency guarantee. The iterator never throws CME.

Key rule: `CopyOnWriteArrayList` is fail-safe by snapshot — always consistent but stale. `ConcurrentHashMap` is fail-safe by weak-consistency — partially reflects concurrent changes. Neither will throw `ConcurrentModificationException`.

Section 2 — Collections Utility Class

2.1 Sorting & Searching

- **Collections.sort(List<T>)** — uses TimSort (stable, $O(n \log n)$); delegates to `list.sort(null)`
- **Collections.sort(List<T>, Comparator<T>)** — TimSort with custom comparator
- **Collections.binarySearch(List, key)** — requires sorted list; returns index if found, or **-(insertion point) - 1** if not found
- Insertion point = index where the key would be inserted to keep sorted order

```
List<Integer> nums = Arrays.asList(1, 3, 5, 7, 9);
int idx = Collections.binarySearch(nums, 5); // returns 2
int notFound = Collections.binarySearch(nums, 4); // returns -3 (insertion point=2, so -(2)-1=-3)
// To get insertion point: -(result) - 1 = -(-3) - 1 = 2
```

2.2 Reordering Methods

Method	Description	Notes
<code>reverse(List)</code>	Reverses the order of elements in place	$O(n)$; uses <code>ListIterator</code> internally
<code>shuffle(List)</code>	Randomly permutes elements using default <code>Random</code>	Uses Fisher-Yates algorithm; $O(n)$
<code>shuffle(List, Random)</code>	Shuffle with specified <code>Random</code> source	Use for reproducible results (seeded <code>Random</code>)
<code>swap(List, i, j)</code>	Swaps elements at positions <code>i</code> and <code>j</code>	$O(1)$ for <code>RandomAccess</code> lists; $O(n)$ for linked
<code>rotate(List, distance)</code>	Rotates elements by distance positions	Positive = rotate right; negative = rotate left

2.3 Bulk Operations

Method	Description
<code>fill(List, obj)</code>	Replaces every element with <code>obj</code> — good for resetting
<code>copy(dest, src)</code>	Copies all elements from <code>src</code> to <code>dest</code> ; <code>dest</code> must be $\geq$ <code>src</code> size
<code>nCopies(n, obj)</code>	Returns immutable <code>List</code> of <code>n</code> copies of <code>obj</code>
<code>frequency(Collection, obj)</code>	Count of elements equal to <code>obj</code> using <code>equals()</code>
<code>disjoint(Collection, Collection)</code>	True if two collections share no common elements — $O(n*m)$ worst case
<code>min(Collection) / max(Collection)</code>	Finds min/max using natural ordering; $O(n)$ linear scan
<code>min(Collection, Comparator) / max(...)</code>	Min/max with custom comparator
<code>addAll(Collection, T...)</code>	Adds all specified elements to collection — varargs version

2.4 Wrapper Factory Methods

- **unmodifiableList/Set/Map/Collection()** — returns an unmodifiable view of the backing collection. Mutations to backing collection are still visible through the view.
- **synchronizedList/Set/Map/Collection()** — wraps in synchronized mutex wrapper. **WARNING:** iteration block MUST be manually synchronized on the returned collection object.
- **checkedList/Set/Map()** — runtime type safety; throws `ClassCastException` on invalid type insertion (guards against heap pollution).

```
// synchronizedList iteration – MUST manually synchronize
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
synchronized (syncList) { // must lock on the syncList itself
    Iterator<String> it = syncList.iterator();
    while (it.hasNext()) { System.out.println(it.next()); }
}
```

```
// unmodifiable view – backing list changes are visible
List<String> backing = new ArrayList<>(Arrays.asList("a", "b"));
List<String> unmod = Collections.unmodifiableList(backing);
backing.add("c"); // allowed
System.out.println(unmod); // prints [a, b, c] – view reflects change
unmod.add("d"); // throws UnsupportedOperationException
```

2.5 Convenience Factory Methods

Method	Description	Mutable?
singleton(obj)	Immutable Set of exactly one element	No
singletonList(obj)	Immutable List of exactly one element	No
singletonMap(k, v)	Immutable Map of exactly one entry	No
emptyList()	Immutable empty List (type-safe, cached singleton)	No
emptySet()	Immutable empty Set (type-safe, cached singleton)	No
emptyMap()	Immutable empty Map (type-safe, cached singleton)	No
nCopies(n, obj)	Immutable List with n copies of obj	No
reverseOrder()	Returns a Comparator that reverses natural order	N/A
reverseOrder(Comparator)	Returns the reverse of the given Comparator	N/A

Prefer `Collections.emptyList()` over `new ArrayList<>()` for empty returns — it avoids allocation and is type-safe (no unchecked cast). Prefer `singletonList()` over `Arrays.asList()` for one element.

Section 3 — Sorting Algorithms in Java

3.1 Arrays.sort() — Dual-Pivot Quicksort (Primitives)

For primitive arrays (`int[]`, `long[]`, `double[]`, etc.), Java uses **dual-pivot Quicksort** (introduced by Yaroslavskiy, Bentley, and Bloch in Java 7). Two pivot elements partition the array into three segments: [less than pivot1], [between pivot1 and pivot2], [greater than pivot2]. This reduces the number of comparisons compared to single-pivot Quicksort.

- Average complexity: $O(n \log n)$; worst case: $O(n^2)$ for adversarial inputs
- NOT stable — equal elements may change relative order
- Uses insertion sort for small arrays ($\text{len} < 47$) for cache efficiency
- Falls back to heapsort for already-seen bad pivot scenarios to guarantee $O(n \log n)$ in practice

```
// Dual-pivot partition concept
// Given array [a0..an], choose pivot1 <= pivot2
// Partition into:
// [a < pivot1] | [pivot1 <= a <= pivot2] | [a > pivot2]
// Recursively sort each part – reduces avg comparisons vs single-pivot
int[] arr = {5, 2, 8, 1, 9, 3};
Arrays.sort(arr); // uses dual-pivot quicksort – NOT stable
// arr = [1, 2, 3, 5, 8, 9]
```

3.2 Arrays.sort(Object[]) — TimSort

For Object arrays (including boxed primitives), Java uses **TimSort** — a hybrid of MergeSort and InsertionSort designed by Tim Peters. Key properties:

- **Stable**: equal elements preserve relative order
- **Adaptive**: exploits existing order in data — $O(n)$ for already-sorted input
- Worst case: $O(n \log n)$; Best case: $O(n)$
- **Runs**: TimSort identifies already-sorted subsequences (runs) — ascending runs are used as-is, descending runs are reversed
- Minimum run size: calculated as ~32-64 elements using formula (`minRunLength`); smaller arrays use pure insertion sort
- **Galloping mode**: when merging, if one run consistently wins, TimSort switches to binary search (galloping) to skip ahead, reducing comparisons

3.3 Collections.sort() and List.sort()

- `Collections.sort(list)` delegates to `list.sort(null)` since Java 8
- `list.sort()` uses `Arrays.sort()` on the underlying array (for `ArrayList`) — TimSort
- `LinkedList.sort()` copies to array, sorts, then writes back — still $O(n \log n)$ but with extra allocation

3.4 Comparator API

```
// Comparator.comparing() chain
List<Employee> employees = getEmployees();
// Sort by department, then by salary descending, then by name
employees.sort(
    Comparator.comparing(Employee::getDepartment)
        .thenComparingDouble(Employee::getSalary).reversed()
        .thenComparing(Employee::getName)
);
// Equivalent verbose form
Comparator<Employee> comp =
    Comparator.comparing(Employee::getDepartment)
        .thenComparing(Comparator.comparingDouble(Employee::getSalary).reversed())
        .thenComparing(Employee::getName);
// Natural order and reverse order
```

```
employees.sort(Comparator.naturalOrder()); // requires Comparable
employees.sort(Comparator.reverseOrder()); // reverse natural order
// Sort Map by value
map.entrySet().stream()
    .sorted(Map.Entry.comparingByValue())
    .collect(Collectors.toMap(...));
```

3.5 Sort Stability Importance

Sort stability means that equal elements maintain their original relative order after sorting. This is critical for multi-key sorts: if you sort by salary first, then by department, a stable sort guarantees that within each department, employees remain in salary order.

TimSort is stable — use it (via `Collections.sort` / `Arrays.sort(Object[])`) whenever sort stability matters. Dual-pivot quicksort for primitives is NOT stable — but since primitives have no identity concept, stability is irrelevant for them.

Section 4 — Collection Complexity Summary Table

Collection	add	get / contains	remove	Iteration	Notes
ArrayList	$O(1)$ amort.	$O(1)$ / $O(n)$	$O(n)$	$O(n)$	Resize copies array; random access $O(1)$
LinkedList	$O(1)$ head/tail	$O(n)$	$O(1)$ with iter	$O(n)$	Double-linked; node allocation overhead
ArrayDeque	$O(1)$ amort.	$O(n)$	$O(1)$ head/tail	$O(n)$	Preferred over Stack/LinkedList for deque
HashMap	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(n)$	$O(n)$ worst with hash collisions; load 0.75
LinkedHashMap	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(n)$	Doubly-linked for insertion/access order
TreeMap	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Red-black tree; sorted keys
HashSet	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(n)$	Backed by HashMap; null key allowed
LinkedHashSet	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(n)$	Backed by LinkedHashMap; insertion order
TreeSet	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Backed by TreeMap; sorted; no null
PriorityQueue	$O(\log n)$	$O(n)$ / $O(1)$ peek	$O(\log n)$	$O(n)$	Min-heap; peek is $O(1)$; no sort guarantee
ConcurrentHashMap	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(n)$	Segment locking; weakly consistent iter
CopyOnWriteArrayList	$O(n)$	$O(1)$ / $O(n)$	$O(n)$	$O(n)$	Array copy on each write; great for rare writes
ConcurrentSkipListMap	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Lock-free sorted concurrent map

$O(1)$ for HashMap/HashSet assumes good hash distribution. Worst case is $O(n)$ when all keys collide. Java 8+ converts long chains (≥ 8 elements) to `TreeNode` structure, capping worst-case to $O(\log n)$. `ArrayList.add()` is $O(1)$ amortized — occasional $O(n)$ resize is amortized across many adds.

Section 5 — Iterator Pattern & Design

5.1 Core Iterator Interfaces

Interface	Key Methods	Purpose
Iterator<E>	hasNext(), next(), remove()	Forward-only iteration; remove() removes last element returned by next()
ListIterator<E>	hasPrevious(), previous(), nextIndex(), previousIndex(), set(), add()	Bidirectional; allows modification during iteration via set() and add()
Iterable<E>	iterator()	Implemented by collections to enable for-each loop; also spliterator()

5.2 How AbstractList Iterator Works

AbstractList provides a private inner class **Itr** implementing Iterator. It maintains cursor (next element index), lastRet (last returned index), and expectedModCount. This pattern is used by ArrayList. LinkedList uses its own node-based iterator.

```
// Simplified AbstractList.Itr internals
private class Itr implements Iterator<E> {
    int cursor = 0; // index of next element to return
    int lastRet = -1; // index of last returned; -1 if none
    int expectedModCount = modCount;

    public boolean hasNext() { return cursor != size(); }

    public E next() {
        checkForComodification(); // modCount != expectedModCount -> throw CME
        int i = cursor;
        if (i >= size()) throw new NoSuchElementException();
        Object[] elementData = ((ArrayList<E>) AbstractList.this).elementData;
        cursor = i + 1;
        return (E) elementData[lastRet = i];
    }

    public void remove() {
        if (lastRet < 0) throw new IllegalStateException();
        checkForComodification();
        try {
            AbstractList.this.remove(lastRet);
            cursor = lastRet;
            lastRet = -1;
            expectedModCount = modCount; // sync after allowed structural change
        } catch (IndexOutOfBoundsException ex) { throw new ConcurrentModificationException(); }
    }
}
```

5.3 Spliterator (Java 8+)

Spliterator is the parallel counterpart to Iterator, designed for parallel stream processing. It can **split** its elements for parallel processing via **trySplit()**.

- **tryAdvance(Consumer)** — processes one element, returns false if none remain
- **forEachRemaining(Consumer)** — processes all remaining elements
- **trySplit()** — splits off a portion for parallel processing; returns null if not splittable
- **estimateSize()** — estimated elements remaining (may be Long.MAX_VALUE if unknown)
- **characteristics()** — bitmask of ORDERED, DISTINCT, SORTED, SIZED, SUBSIZED, IMMUTABLE, CONCURRENT, NONNULL

Characteristic	Meaning	Example Collections
ORDERED	Elements have a defined encounter order	ArrayList, LinkedHashMap, arrays

DISTINCT	No duplicate elements	HashSet, TreeSet
SORTED	Elements are in sorted order	TreeSet, TreeMap
SIZED	estimateSize() is exact	ArrayList, HashMap, arrays
SUBSIZED	Child spliterators are also SIZED	Arrays, ArrayList
IMMUTABLE	Source cannot be structurally modified	List.of(), arrays after asList
CONCURRENT	Source can be modified concurrently	ConcurrentHashMap
NONNULL	No null elements	TreeSet, HashMap keys (null ok in HashMap)

```
// Spliterator for parallel stream – ArrayList example
List<Integer> list = new ArrayList<>(Arrays.asList(1,2,3,4,5,6,7,8));
Spliterator<Integer> s1 = list.spliterator(); // covers [0, 8)
Spliterator<Integer> s2 = s1.trySplit(); // s2 covers [0,4), s1 covers [4,8)
// Each can be processed in parallel by ForkJoinPool
s1.forEachRemaining(System.out::println); // 5,6,7,8
s2.forEachRemaining(System.out::println); // 1,2,3,4
```

Section 6 — Java 9+ Immutable Collection Factories

6.1 List.of(), Set.of(), Map.of()

- Introduced in Java 9; produce **truly immutable** collections (not just unmodifiable views)
- **No null elements allowed** — NullPointerException on null input
- Set.of() throws IllegalArgumentException on duplicate elements
- Iteration order is not guaranteed for Set.of() and Map.of()
- Thread-safe for reads (immutable); no need for synchronization

```
// List.of() – immutable, no nulls
List<String> list = List.of("a", "b", "c");
list.add("d"); // throws UnsupportedOperationException
list.set(0, "z"); // throws UnsupportedOperationException

// Set.of() – immutable, no nulls, no duplicates
Set<Integer> set = Set.of(1, 2, 3, 4, 5);

// Map.of() – up to 10 entries (key-value pairs as alternating args)
Map<String, Integer> map = Map.of("a", 1, "b", 2, "c", 3);

// Map.ofEntries() – for more than 10 entries
Map<String, Integer> bigMap = Map.ofEntries(
    Map.entry("a", 1),
    Map.entry("b", 2),
    // ... unlimited entries
    Map.entry("z", 26)
);

// Defensive copies – creates new immutable from existing collection
List<String> immutableCopy = List.copyOf(existingList); // new immutable list
Set<String> immutableSet = Set.copyOf(existingSet);
Map<String, Integer> immutableMap = Map.copyOf(existingMap);
```

6.2 Performance Optimizations

Java 9+ collection factories use specialized internal implementations optimized for small sizes:

Size	List.of() Implementation	Notes
0 elements	ImmutableCollections.EMPTY_LIST singleton	No allocation at all
1 element	ImmutableCollections\$List12 (single field)	Single object field; ultra-compact
2 elements	ImmutableCollections\$List12 (two fields)	Two object fields; no array overhead
3+ elements	ImmutableCollections\$ListN (Object[] array)	Array-backed; still immutable

6.3 Comparison: Factory vs Collections.unmodifiable*

Feature	List.of() / Collections.Factory	Collections.unmodifiableList()
Creates new object?	YES — new immutable object	NO — view of existing
Backing mutable list?	None — truly immutable	Yes — mutations visible through view
Null elements?	Not allowed — NPE	Allowed (if backing list has nulls)
Thread safety?	Yes (immutable)	Only if backing list is thread-safe
Type?	ImmutableCollections.* (internal)	Collections.UnmodifiableList
Use case?	Return values, constants, safe sharing	Prevent external modification of internal list

Rule of thumb: Use List.of() when creating a new immutable collection (constants, test data, return values). Use Collections.unmodifiableList() when you want to expose a view of an internal mutable list without exposing mutability.

Section 7 — Common Anti-Patterns & Best Practices

7.1 Performance Anti-Patterns

Anti-Pattern	Problem	Fix
ArrayList for membership test (contains())	O(n) per check — catastrophic in loops	Use HashSet — O(1) average contains()
HashMap with enum keys	HashMap overhead for enum keys	Use EnumMap — array-backed, O(1), less memory
Not pre-sizing collections	Multiple array resizes, each O(n)	new ArrayList<>(expectedSize) or HashMap(size, 0.75f)
List.remove(int) confusion	remove(int) removes by index, not value	Cast: list.remove(Integer.valueOf(5)) to remove by value
Using new ArrayList<>() for empty return	Unnecessary allocation	Return Collections.emptyList() or List.of()
Collections.EMPTY_LIST (raw type)	Unchecked cast warnings	Use Collections.emptyList() — type-safe generic version

7.2 Concurrency Anti-Patterns

```
// ANTI-PATTERN: synchronizedList iteration without sync
List<String> list = Collections.synchronizedList(new ArrayList<>());
// In another thread: list.add("x") – may cause CME!
for (String s : list) { process(s); } // NOT thread-safe!
// FIX: synchronize on the list object during iteration
synchronized (list) {
    for (String s : list) { process(s); }
}
// BETTER: use CopyOnWriteArrayList for read-heavy scenarios
List<String> cowList = new CopyOnWriteArrayList<>();
for (String s : cowList) { process(s); } // safe – iterates over snapshot
```

7.3 Mutation Anti-Patterns

```
// ANTI-PATTERN: Collection.remove() during for-each loop
List<String> list = new ArrayList<>(Arrays.asList("a","b","c"));
for (String s : list) {
    if (s.equals("b")) list.remove(s); // ConcurrentModificationException!
}
// FIX 1: iterator.remove()
Iterator<String> it = list.iterator();
while (it.hasNext()) { if (it.next().equals("b")) it.remove(); }
// FIX 2: removeIf (Java 8+) – cleanest
list.removeIf("b"::equals);
// FIX 3: collect then removeAll
List<String> toRemove = list.stream().filter("b"::equals).collect(toList());
list.removeAll(toRemove);
```

7.4 API Design Best Practices

- **Defensive copy:** public setters/constructors receiving collections should copy them: `this.items = new ArrayList<>(items)`
- **Immutable returns:** public getters returning collections should return `List.copyOf(internalList)` or `Collections.unmodifiableList()`
- **Mutable keys:** NEVER use mutable objects as HashMap keys — if the key's `hashCode()` changes, the entry is lost
- **String keys:** Fine as HashMap keys — String is immutable with cached `hashCode`

- **computeIfAbsent:** Replace null-checking anti-pattern with `map.computeIfAbsent(key, k -> new ArrayList<>()).add(value)`

```
// ANTI-PATTERN: null check pattern
if (!map.containsKey(key)) map.put(key, new ArrayList<>());
map.get(key).add(value);
// CLEAN: computeIfAbsent
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value);
// ANTI-PATTERN: mutable object as key
class MutableKey { int id; }
Map<MutableKey, String> map = new HashMap<>();
MutableKey key = new MutableKey(1);
map.put(key, "value");
key.id = 2; // hashCode changed! map.get(key) returns null – entry lost!
// ANTI-PATTERN: returning internal list
public List<Order> getOrders() { return orders; } // caller can modify!
// FIX:
public List<Order> getOrders() { return Collections.unmodifiableList(orders); }
```

Section 8 — Collection Selection Decision Guide

8.1 Decision Table

Requirement	Best Choice	Reason
Ordered list with fast random access	ArrayList	O(1) get; contiguous memory
Frequent insertions/deletions at ends	ArrayDeque	O(1) at both ends; better than LinkedList
Stack (LIFO)	ArrayDeque (as stack)	Faster than java.util.Stack (synchronized)
Queue (FIFO)	ArrayDeque / LinkedList	ArrayDeque preferred; no null needed
Priority-based processing	PriorityQueue	Min-heap; O(log n) offer/poll
Fast key lookup (unordered)	HashMap	O(1) avg get/put; load factor 0.75
Key lookup preserving insertion order	LinkedHashMap	HashMap + doubly-linked list
Key lookup with sorted keys	TreeMap	Red-black tree; O(log n)
Unique elements, unordered	HashSet	Backed by HashMap
Unique elements, sorted	TreeSet	Backed by TreeMap; navigable
Thread-safe key-value store	ConcurrentHashMap	Segment locking; high concurrency
Thread-safe sorted set	ConcurrentSkipListSet	Lock-free; sorted; concurrent
Read-heavy thread-safe list	CopyOnWriteArrayList	Array snapshot on write; safe iteration
Enum keys	EnumMap	Array-backed; O(1); compact; faster than HashMap
Enum values as set	EnumSet	Bit-vector backed; O(1) all ops; very fast
Immutable collection (constants)	List.of() / Set.of() / Map.of()	Java 9+; true immutability; null-free
Prevent modification of internal list	Collections.unmodifiableList()	View; backing changes still visible

8.2 Arrays.asList vs List.of vs new ArrayList

Method	Mutable (add/remove)?	Mutable (set)?	Null Elements?	Backed By
Arrays.asList(a,b,c)	NO — fixed size	YES	Allowed	Original array (shared)
List.of(a,b,c)	NO	NO	NO — NPE	Internal immutable impl
new ArrayList<>(list)	YES	YES	Allowed	New array copy
Collections.singletonList(a)	NO	NO	Allowed	Single-element immutable

8.3 Memory Considerations

- Primitives in collections must be boxed — int becomes Integer (16 bytes object header + 4 bytes value)
- For primitive-heavy collections, consider Eclipse Collections (IntArrayList) or Trove — avoid boxing overhead
- ArrayList over-allocates: initial capacity 10, grows by 50% each resize
- HashMap default initial capacity 16, load factor 0.75 — resize at 12 entries
- LinkedList: each node = 3 object references (prev, next, element) + object header — ~48 bytes overhead per element
- ArrayDeque: more memory-efficient than LinkedList for queue/stack use cases

Q&A; — Standard Questions (Q1–Q35)

Q1. How does modCount work to implement fail-fast iterators?

Every structural modification (add, remove, clear) on an `AbstractList` or `AbstractMap` increments the `modCount` field. When an iterator is created, it saves `expectedModCount = modCount`. Each call to `next()` or `remove()` checks if `modCount == expectedModCount`. If they differ, a `ConcurrentModificationException` is thrown immediately. This is best-effort detection — not guaranteed (e.g., if exactly one add and one remove happen between checks, `modCount` could accidentally match `expectedModCount`).

Q2. Is `ConcurrentHashMap`'s iterator fail-fast or fail-safe?

`ConcurrentHashMap`'s iterator is fail-safe with weak-consistency semantics. It never throws `ConcurrentModificationException`. It captures a reference to the internal `Node[]` table at construction time and traverses it following actual next pointers. Elements present at creation time will be seen; elements inserted or removed after creation may or may not be reflected.

Q3. What is weak-consistency in concurrent collection iterators?

Weak-consistency means the iterator reflects some — but not necessarily all — concurrent modifications. For `ConcurrentHashMap`: elements that existed when the iterator was created are guaranteed to be seen. Newly added elements may or may not be seen. Removed elements may still appear. This is different from a full snapshot (like `CopyOnWriteArrayList`) or a full view (like a synchronized collection).

Q4. When does `Collections.binarySearch()` return a negative value?

It returns a negative value when the key is not found in the list. The return value is $-(\text{insertion point}) - 1$, where insertion point is the index at which the key would be inserted to maintain sorted order. To recover the insertion point: $\text{insertionPoint} = -(\text{result}) - 1$. Example: search for 4 in `[1,3,5,7]` returns -3 (insertion point = 2, so $-(2)-1 = -3$).

Q5. How is TimSort different from MergeSort?

TimSort is an adaptive hybrid of MergeSort and InsertionSort. Key differences: (1) TimSort identifies already-sorted subsequences (runs) in the data and merges them, while MergeSort always divides blindly. (2) TimSort uses insertion sort for small runs (< 64 elements). (3) TimSort employs galloping mode when merging — if one run consistently has smaller elements, it switches to binary search to skip ahead. (4) TimSort is $O(n)$ for already-sorted data; MergeSort is always $O(n \log n)$. Both are stable.

Q6. Why is `Arrays.sort()` for primitives unstable (quicksort) but stable for objects (timsort)?

Stability only matters when equal elements have distinguishable identity. Primitive values have no identity — two int values of 5 are completely interchangeable. So stability is meaningless for primitives. Quicksort (dual-pivot) offers better cache performance and average-case speed for primitives. For objects, stability is crucial (e.g., multi-key sorting), so TimSort is used despite being slightly slower in practice. Object arrays also avoid the worst-case $O(n^2)$ of quicksort.

Q7. What does `Collections.shuffle()` do internally and why does it use `Random`?

`Collections.shuffle()` implements the Fisher-Yates algorithm (Knuth shuffle): starting from the last element, it swaps each element with a randomly chosen element at or before its current position. Time complexity: $O(n)$. It uses `Random` (or a provided `Random`) to generate swap indices. A seeded `Random` produces reproducible shuffles — useful for testing. If the list is not `RandomAccess` (e.g., `LinkedList`), it copies elements to an array, shuffles, then writes back.

Q8. What is the difference between `Collections.synchronizedList()` and `CopyOnWriteArrayList` for thread safety?

`synchronizedList` wraps every method in a synchronized block using the list as a mutex — fine for individual operations but iteration requires external synchronization. Every read and write is synchronized, so readers block writers. `CopyOnWriteArrayList` makes a full array copy on every write; reads never block and iterate over a snapshot. `CopyOnWriteArrayList` is ideal for read-heavy, write-rare scenarios (e.g., listener lists). `synchronizedList` is better when writes are frequent and iteration is rare.

Q9. How do you safely remove elements while iterating? Name all 3 safe methods.

Method 1 — `iterator.remove()`: Use explicit iterator, call `it.remove()` after `it.next()`. Allowed because `it.remove()` updates `expectedModCount` internally. Method 2 — `removeIf()` (Java 8+): `list.removeIf(predicate)` — internally uses `iterator.remove()`. Cleanest option. Method 3 — collect-then-remove: collect elements to remove in a separate list, then call `list.removeAll(toRemove)` after iteration completes. NEVER call `collection.remove(element)` directly inside a for-each loop — causes `ConcurrentModificationException`.

Q10. What is `Splitter` and how does it enable parallel streams?

`Splitter` (`Splitable Iterator`) is a Java 8 interface for traversing and partitioning elements. `trySplit()` divides the element range into two `Splitter`s, enabling recursive decomposition for the `ForkJoinPool`. `Stream.parallel()` obtains a `Splitter` from the source and recursively calls `trySplit()` until chunks are small enough, then processes each chunk in a `ForkJoin` task. The `characteristics()` bitmask (`SIZED`, `ORDERED`, etc.) allows the stream engine to optimize processing.

Q11. What is the difference between `List.of()` and `Collections.unmodifiableList()`?

`List.of()` creates a new, truly immutable list — no null elements allowed, iteration order guaranteed, completely independent of any source. `Collections.unmodifiableList()` creates an unmodifiable VIEW of an existing mutable list — nulls are allowed if the backing list has them, and mutations to the backing list are still visible through the unmodifiable view (it just prevents the caller from modifying via the view reference).

Q12. When would you use `Collections.frequency()`?

`Collections.frequency(collection, obj)` counts elements equal to `obj` (using `equals()`). Useful for: counting occurrences in a List without converting to a stream; quick frequency check for a known value; validation (asserting no duplicates). Time complexity: $O(n)$ linear scan. For multiple frequencies, prefer building a Map via `Collectors.groupingBy(Function.identity(), Collectors.counting())`.

Q13. What does `Collections.disjoint()` do and what is its time complexity?

`Collections.disjoint(c1, c2)` returns true if the two collections have no elements in common — i.e., their intersection is empty. Implementation: iterates over `c1` and checks if `c2.contains(each)`. Time complexity: $O(n)$ if `c2` is a `HashSet/TreeSet` ($O(1)/O(\log n)$ contains), or $O(n*m)$ if `c2` is a List. For best performance, pass the larger collection as `c1` and the collection with fast `contains()` as `c2`.

Q14. Why should you prefer `Collections.emptyList()` over `new ArrayList<>()`?

`Collections.emptyList()` returns a cached singleton — `EMPTY_LIST` — avoiding any object allocation. It is type-safe via generics (no unchecked cast), immutable (safe to share), and signals intent clearly: this is guaranteed to be empty. `new ArrayList<>()` allocates a new object with initial capacity 10 (pre-Java 8) or a shared empty array (Java 8+). For method return values meaning 'no results', `emptyList()` is the canonical choice. Prefer `List.of()` in Java 9+.

Q15. How does `Comparator.comparing().thenComparing()` work internally?

`Comparator.comparing(keyExtractor)` creates a `Comparator` that applies `keyExtractor` to both arguments and compares the resulting keys. `thenComparing(secondComparator)` creates a new `Comparator` that first delegates to the first comparator; if it returns 0 (equal), it delegates to `secondComparator`. Internally, each `thenComparing()` call wraps the previous `Comparator` in a new lambda/anonymous instance: `(a, b) -> { int r = first.compare(a,b); return r != 0 ? r : second.compare(a,b); }`. Multiple `thenComparing()` calls create a chain of these wrappers.

Q16. What is sort stability and why does it matter for multi-field sorting?

A sort is stable if equal elements preserve their relative order from the input. This matters for multi-field sorting because it enables incremental sorting: sort by the least significant field first, then by the most significant. The stability guarantee ensures the first sort's order is preserved among ties in the second sort. Without stability, you must use a single multi-key `Comparator` with `thenComparing()` instead of sequential sorts.

Q17. How do you sort a Map by value?

Option 1 — Stream: `map.entrySet().stream().sorted(Map.Entry.comparingByValue()).collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue, (e1,e2)->e1, LinkedHashMap::new))`. Option 2 — List sort: `List<Map.Entry> entries = new ArrayList<>(map.entrySet()); entries.sort(Map.Entry.comparingByValue());` Option 3 — TreeMap with value comparator (unreliable — TreeMap sorts by key, not value). The Stream + LinkedHashMap approach is most idiomatic and preserves insertion order.

Q18. What is the time complexity of Collections.sort() and what algorithm does it use?

`Collections.sort()` uses TimSort — $O(n \log n)$ worst case, $O(n)$ best case (already sorted), $O(n \log n)$ average. Since Java 8, `Collections.sort(list)` delegates to `list.sort(null)`, which for `ArrayList` calls `Arrays.sort(Object[])` — TimSort. It is stable. Minimum guaranteed: $O(n \log n)$ for any input. Memory: $O(n)$ auxiliary space for merge operations.

Q19. How do you get a thread-safe sorted set in Java?

Use `ConcurrentSkipListSet` — a concurrent, thread-safe, sorted set based on a skip list structure. It implements `NavigableSet` and `SortedSet`. All operations (add, remove, contains) are $O(\log n)$. It uses a lock-free algorithm (CAS operations) for high concurrency without blocking. Alternative: `Collections.synchronizedSortedSet(new TreeSet<>())` — but this blocks all concurrent access. `ConcurrentSkipListSet` is preferred for high-concurrency scenarios.

Q20. When is removeIf() more efficient than iterator-based removal?

`removeIf()` is more efficient for `ArrayList` because its `ArrayList`-specific override avoids multiple shifts. Standard iterator removal from `ArrayList` shifts elements for each removal ($O(n)$ per removal, $O(n^2)$ total). `ArrayList.removeIf()` uses a bitmask approach: marks elements to remove in one pass, then compacts the array in a single $O(n)$ operation. For `LinkedList`, `iterator.remove()` is $O(1)$ per element and `removeIf()` has similar performance.

Q21. What happens if you call iterator.remove() without calling next() first?

It throws `IllegalStateException`. The iterator's `lastRet` field (index of last returned element) is initialized to -1. `remove()` checks if `lastRet < 0` and throws `IllegalStateException` if so. This prevents removing an element that was never returned, or removing twice after a single `next()`. After `remove()` succeeds, `lastRet` is reset to -1, so calling `remove()` again without `next()` also throws.

Q22. Can you use ListIterator to add elements during iteration?

Yes — `ListIterator.add(e)` inserts the element immediately before the next element that would be returned by `next()`. The added element is NOT returned by the next subsequent call to `next()` or `previous()`. Crucially, `add()` via `ListIterator` does NOT throw `ConcurrentModificationException` — it updates `expectedModCount` after the structural modification, keeping the iterator valid. This is the only safe way to insert elements during forward iteration.

Q23. What is the difference between Set.of() and new HashSet<>(Arrays.asList(...))?

`Set.of()` produces a truly immutable set with guaranteed no-null contract, no duplicate contract (throws `IAE` on construction), and an unspecified iteration order. `new HashSet<>(Arrays.asList(...))` produces a mutable `HashSet` that allows null, silently ignores duplicates, and has `HashMap`-bucket iteration order. For read-only constant sets, `Set.of()` is more efficient (specialized small-size implementations) and safer (immutability enforced at API level).

Q24. What does Collections.nCopies() return and when is it useful?

`Collections.nCopies(n, obj)` returns an immutable `List` of n references to the same object `obj` — NOT n separate copies. It's backed by a single object reference repeated logically (very memory-efficient). Useful for: initializing a `List` to a known size with a default value; as input to `addAll()`; filling a matrix row with zeros. Since all elements are the same reference, mutation of `obj` affects all 'copies'. For primitives (which are immutable), this is safe.

Q25. How does EnumMap differ from HashMap for enum keys?

`EnumMap` is backed by an array indexed by the enum's ordinal — $O(1)$ all operations with no hashing, no collision, and no boxing overhead. Memory usage is proportional to the number of enum constants, not the number of entries. Iteration is in enum declaration order. `EnumMap` is significantly faster and more memory-efficient than `HashMap` for enum keys. It does not allow null keys (throws `NPE`).

Q26. What is the difference between ArrayList and Vector?

Both are resizable array implementations but Vector is synchronized (every method is synchronized) while ArrayList is not. Vector is a legacy class from Java 1.0; ArrayList was added in Java 1.2 with the Collections Framework. Vector grows by 100% (doubles) by default; ArrayList grows by 50%. For thread safety, prefer CopyOnWriteArrayList or synchronized ArrayList over Vector. Vector is considered obsolete.

Q27. Why is LinkedList generally slower than ArrayList in practice?

Despite $O(1)$ insertions at head/tail, LinkedList is often slower due to: (1) poor cache locality — nodes are scattered in heap memory, causing cache misses on traversal; (2) each node requires 3 object references (prev, next, element) plus object header (~48 bytes overhead per element vs ~4 bytes in ArrayList); (3) random access is $O(n)$; (4) iterator traversal triggers many cache misses. ArrayList's contiguous memory layout makes CPU prefetching effective.

Q28. What is the load factor in HashMap and what is its impact?

Load factor is the ratio threshold at which HashMap resizes its bucket array. Default is 0.75. When $\text{size} > \text{capacity} * \text{loadFactor}$, the map doubles its capacity and rehashes all entries — $O(n)$ operation. Lower load factor (e.g., 0.5) reduces collisions but wastes memory and triggers more frequent resizes. Higher load factor (e.g., 0.9) saves memory but increases collisions and degrades to $O(n)$ in extreme cases. 0.75 is the empirically optimal balance between time and space.

Q29. How does PriorityQueue work and what are its limitations?

PriorityQueue is a min-heap backed by an array. The smallest element (per natural order or Comparator) is at the root (index 0) — `peek()` returns it in $O(1)$. `offer()/add()` inserts and sifts up in $O(\log n)$. `poll()` removes the root and sifts the last element down in $O(\log n)$. Limitations: (1) only `peek/poll` gives the minimum in $O(1)$ — arbitrary element access is $O(n)$; (2) NOT thread-safe; use `PriorityBlockingQueue` for concurrent use; (3) does not support $O(1)$ decrease-key.

Q30. What is the difference between Iterator.remove() and List.remove() during iteration?

`Iterator.remove()` removes the last element returned by `next()` from the underlying collection AND updates `expectedModCount` to match `modCount`, keeping the iterator valid. `List.remove()` directly modifies the collection, incrementing `modCount` WITHOUT updating the iterator's `expectedModCount` — the next call to `iterator.next()` will throw `ConcurrentModificationException`. Always use `iterator.remove()` (or `removeSelf()`) when removing during iteration.

Q31. How do you implement a least-recently-used (LRU) cache using Java collections?

Use `LinkedHashMap` with access-order mode: `new LinkedHashMap<>(capacity, 0.75f, true)`. Override `removeEldestEntry()`: `protected boolean removeEldestEntry(Map.Entry eldest) { return size() > maxCapacity; }`. With `access-order=true`, `get()` and `put()` move the accessed entry to the tail. `removeEldestEntry()` is called after each `put()` — return true when size exceeds capacity to evict the least-recently-used (`eldest = head`) entry automatically.

Q32. What is TreeMap.floorKey() and ceilingKey() used for?

`TreeMap` implements `NavigableMap`, providing methods to find keys relative to a given key. `floorKey(k)` returns the greatest key less than or equal to `k` (or null). `ceilingKey(k)` returns the smallest key greater than or equal to `k` (or null). `higherKey(k)` returns strictly greater; `lowerKey(k)` returns strictly less. These are useful for range queries, interval trees, event scheduling, and finding nearest neighbors in sorted data — all in $O(\log n)$ time.

Q33. How does Deque differ from Queue and when would you use ArrayDeque?

Queue is a one-ended structure (add to tail, remove from head — FIFO). Deque (Double-Ended Queue) supports insertion and removal at both ends. `ArrayDeque` implements Deque with a circular array — $O(1)$ amortized at both ends. Use `ArrayDeque` as: (1) a Stack (push/pop at head) — faster than Stack; (2) a Queue (offer at tail, poll at head) — faster than LinkedList; (3) a sliding window buffer in algorithms. `ArrayDeque` does NOT support null elements.

Q34. What happens when two keys in HashMap have the same hashCode?

They collide and are placed in the same bucket. Pre-Java 8: stored as a linked list — $O(n)$ for get/put in the worst case (all keys same hash). Java 8+: when a bucket's linked list grows to 8 or more entries, it's converted to a `TreeNode` (red-black tree) — reducing worst case to $O(\log n)$. When a bucket shrinks to 6 entries (after removals), it reverts to linked list. The bin is treeified only if total capacity ≥ 64 .

Q35. What is the difference between `HashMap.get()` returning null and `containsKey()` returning false?

`HashMap` allows null values. `map.get(key)` returning null could mean: (1) the key doesn't exist, OR (2) the key exists with a null value. To distinguish: use `map.containsKey(key)`. If `containsKey()` returns true but `get()` returns null, the value is explicitly null. This distinction matters in code that must handle both cases differently, e.g., `map.getDefault(key, defaultValue)` still returns null if value is explicitly null.

Q&A; — Advanced Questions (Q36–Q52)

[ADV] Q36. TimSort internals: what are 'runs' and how does TimSort exploit existing order? Explain minimum run size calculation and galloping mode.

A 'run' is a maximal already-sorted subsequence. TimSort scans the input to identify runs — ascending sequences are used directly; descending sequences are reversed. This makes TimSort adaptive: $O(n)$ when input is already sorted (one run, no merging). Minimum run size (`minRun`) is computed as: find the largest k in $[32, 64]$ such that n/k is a power of 2 minus 1 — ensures merge balance. Arrays smaller than `minRun` use binary insertion sort. Galloping mode: during merge, if the same run 'wins' consecutively (default threshold: 7 times), TimSort switches from element-by-element comparison to binary search within the winning run, skipping large chunks — dramatically faster for nearly-sorted data with large sorted segments.

[ADV] Q37. Dual-pivot Quicksort: how do two pivots partition into 3 segments and why is it better than single-pivot for typical data?

The algorithm selects two pivots $p1 \leq p2$, then partitions into three regions: $[e < p1]$, $[p1 \leq e \leq p2]$, $[e > p2]$. This is done with a single pass using three pointers (`less`, `great`, `k`). Single-pivot quicksort creates 2 partitions per level, requiring $\log_2(n)$ levels. Dual-pivot creates 3 partitions per level, requiring $\log_3(n)$ levels — about $1/\log_2(3) \approx 0.63\times$ fewer levels. In practice, the average number of comparisons is reduced by ~20-25% for random data. Java's implementation (Yaroslavskiy, 2009) also uses insertion sort for small segments and has careful pivot selection to avoid worst-case $O(n^2)$ for sorted/reverse-sorted inputs.

[ADV] Q38. `Splitter.trySplit()` contract: describe how `ArrayList`'s `Splitter` uses this to enable parallel streams.

After `trySplit()`, the original `Splitter` covers $[mid, end)$ and the returned `Splitter` covers $[start, mid)$. `ArrayList` uses `ArrayListSplitter`, which stores `fence` (exclusive end index) and `index` (current position). `trySplit()` computes `mid = (index + fence) >>> 1`; returns a new `ArrayListSplitter` (`index`, `mid`), and sets `index = mid`. The original now starts at `mid`. The `ForkJoinPool` recursively calls `trySplit()` until chunks reach a parallelism threshold (typically ~1024 elements), then assigns each chunk to a worker thread. `SIZED` + `SUBSIZED` characteristics allow the framework to compute exact work distribution without estimating.

[ADV] Q39. How could `modCount` integer overflow (wrap-around) theoretically defeat the fail-fast mechanism? Why is this acceptable in practice?

`modCount` is an `int`. After $2^{31} - 1$ structural modifications, it wraps to -2^{31} and continues incrementing. If an iterator captures `expectedModCount` at value X , and exactly 2^{32} modifications happen before the next `next()` call, `modCount` wraps back to X — the check passes even though 2^{32} modifications occurred. This is acceptable because: (1) 2^{32} modifications in the window between iterator creation and `next()` is effectively impossible in practice; (2) fail-fast is documented as best-effort, not guaranteed (JDK Javadoc states it cannot be relied upon for correctness — it's a debugging aid); (3) the contract explicitly says `ConcurrentModificationException` should not be caught and acted upon.

[ADV] Q40. Weak-consistency of `ConcurrentHashMap`'s iterator: how it uses the `Node[]` table snapshot and follows actual next pointers.

`ConcurrentHashMap.HashIterator` captures the `Node[]` table reference (not a copy) at construction time. It iterates bucket by bucket (index from 0 to `table.length`). For each non-null bucket, it follows `Node.next` pointers to traverse the chain. Since the table array reference itself is captured (not copied), structural resizes that replace the table with a larger array are NOT reflected. However, since nodes are linked by reference (not by position), a removed node may still be reachable via old next pointers while a new insertion into a bucket may or may not be seen depending on whether that bucket has been iterated yet. This is the precise mechanism of weak-consistency.

[ADV] Q41. `AbstractList.hashCode()` implementation: the polynomial 31-based rolling hash — why this specific algorithm was standardized in JLS.

`AbstractList.hashCode()` computes: `int hashCode = 1; for (E e : this) hashCode = 31 * hashCode + (e == null ? 0 : e.hashCode());`. The multiplier 31 was chosen because: (1) it is an odd prime — odd multipliers avoid losing bits from even-multiplier right-shifts; (2) $31 * i = (i < 5) - i$ — JVMs can optimize this to a shift+subtract; (3) large enough to spread values over a wide range, small enough to avoid overflow patterns that cluster values. The rolling polynomial form (Horner's method) gives good avalanche behavior — changing any one element changes the final hash significantly. This is standardized in JLS 9.6.4.4 to ensure List equality contract (equal lists must have equal hashCodes).

[ADV] Q42. `Collections.unmodifiableList()` internals: delegation and how `iterator()` creates an `unmodifiableIterator`.

`Collections.UnmodifiableList` stores a private final `List c` (the backing list). Every read method delegates: `size()` -> `c.size()`, `get(i)` -> `c.get(i)`, `contains(o)` -> `c.contains(o)`, etc. Every mutating method throws `UnsupportedOperationException` directly (`add`, `set`, `remove`, `clear`, etc.). `iterator()` returns new `UnmodifiableIterator(c.iterator())`, which wraps the backing iterator: `hasNext()` and `next()` delegate to the backing iterator, but `remove()` throws `UnsupportedOperationException`. `listIterator()` similarly wraps in `UnmodifiableListIterator`, blocking `set()` and `add()`. Since the backing list reference `c` is directly used, any structural changes to `c` are visible through the unmodifiable wrapper — it is a live view, not a snapshot.

[ADV] Q43. Memory overhead of `synchronizedList`: `SynchronizedList` wrapper and its mutex object.

`Collections.SynchronizedList` stores two fields: the backing list `c` and a mutex object (final `Object mutex`). By default, `mutex = this` (the `SynchronizedList` itself). Every method synchronizes on `mutex`: `synchronized (mutex) { return c.size(); }`. Object overhead: `SynchronizedList` has the standard object header (12-16 bytes on 64-bit JVM with compressed OOPs), plus two object reference fields (`c` and `mutex`) = ~32 bytes total wrapper overhead. The mutex object itself is the `SynchronizedList` instance (no additional object). However, iteration is NOT covered by any method-level sync — external synchronization: `synchronized (list) { ... }` is required.

[ADV] Q44. Why do `List.of()` implementations use package-private `ImmutableCollections` classes? What is the Jigsaw module boundary reason?

Java 9 introduced the module system (Project Jigsaw). `List.of()` returns instances of package-private classes like `ImmutableCollections.List12`, `ImmutableCollections.ListN` inside the `java.base` module. These are implementation details hidden by the module boundary — external code cannot reference them by name, cannot subclass them, and cannot use reflection to modify them (module encapsulation). This enforces true immutability at the module level. The multiple specialized classes (`List0`, `List1`, `List12`, `ListN`) optimize memory: `List12` uses two direct fields instead of an array, eliminating array header and bounds-check overhead for 1-2 element lists.

[ADV] Q45. `Comparator.comparing()` chain: how each `thenComparing()` creates a new wrapping `Comparator` via composition.

`Comparator.comparing(keyExtractor)` returns a lambda: `(a,b) -> keyExtractor.apply(a).compareTo(keyExtractor.apply(b))`. `thenComparing(other)` is defined in the `Comparator` interface as a default method: `return (a,b) -> { int res = this.compare(a,b); return res != 0 ? res : other.compare(a,b); }`. Each `thenComparing()` creates a new `Comparator` that closes over the previous one (`this`) and the new one. Multiple chaining creates a linked list of `Comparator` wrappers. This is the Decorator pattern applied to function composition. The chain is evaluated left-to-right, short-circuiting when a non-zero result is found. `reversed()` wraps the whole chain: `(a,b) -> -this.compare(a,b)`.

[ADV] Q46. Adaptive behavior of TimSort: why is it $O(n)$ for already-sorted input? Walk through the algorithm for a sorted array.

For a fully sorted array `[1,2,3,...,n]`: (1) TimSort scans from left, detecting a run starting at index 0. Since elements are ascending, the run extends to the last element — the entire array is one run of length `n`. (2) TimSort pushes this run onto its merge stack. (3) Since there's only one run, the merge condition (stack balance invariant) is never triggered — no merges happen. (4) Result: $O(n)$ scan + 0 merges = $O(n)$. Similarly, a fully reverse-sorted array creates one run (`reversed` in $O(n)$), then no merges. For `k` runs of equal length, TimSort does $O(n \log k)$ work — optimal for `k`-sorted data.

[ADV] Q47. How do you build a memory-efficient multiset (element frequency map) with proper encapsulation?

A multiset tracks element frequencies. Implementation: use a `HashMap` or `HashMap` internally. Encapsulate with: `add(T e) { map.merge(e, 1, Integer::sum); }, remove(T e) { map.merge(e, -1, Integer::sum); if (map.get(e) <= 0) map.remove(e); }, count(T e) { return map.getOrDefault(e, 0); }, elementSet() { return Collections.unmodifiableSet(map.keySet()); }, }` For thread-safety: use `ConcurrentHashMap` with `computeIfPresent/merge`. For memory efficiency: use `HashMap` with capacity pre-sized to expected distinct elements. Java's Guava library provides `HashMultiset` as a production-ready implementation.

[ADV] Q48. What is the internal structure of `TreeMap` and how does it maintain sorted order?

`TreeMap` is backed by a **red-black tree** — a self-balancing binary search tree with the property that no path from root to leaf is more than twice as long as any other (colored red/black). Each `Entry` node has: key, value, left, right, parent references, and a color bit. Insertions and deletions trigger rotations and recolorings to maintain balance. In-order traversal (left → root → right) yields keys in sorted order — this is why iteration is in sorted key order. All operations (`put`, `get`, `remove`, `firstKey`, `lastKey`, `subMap`) are $O(\log n)$. Requires keys to be `Comparable` or a `Comparator` to be provided at construction.

[ADV] Q49. How does `LinkedHashMap` implement access-order vs insertion-order mode?

`LinkedHashMap` extends `HashMap` and maintains a doubly-linked list running through all entries. Each `Entry` has before and after pointers in addition to `HashMap`'s next pointer. In **insertion-order mode** (default): the linked list order reflects insertion order; accessing (`get`) an entry does not change its position. In **access-order mode** (`accessOrder=true`): every `get()` and `put()` moves the accessed entry to the tail of the doubly-linked list, making the head always the least-recently-used entry. The `removeEldestEntry()` hook (called after `put()`) enables automatic LRU eviction. Overhead: two extra references per `Entry` vs `HashMap`'s single next.

[ADV] Q50. What is the `ConcurrentHashMap` segmentation/stripped locking model in Java 8+ vs Java 7?

Java 7: `ConcurrentHashMap` divided the table into 16 Segments (sub-`HashMap`s), each with its own lock. Concurrency level = 16 by default — 16 threads could write simultaneously to different segments. Java 8+: Segments were eliminated. Instead, Java 8 uses **node-level CAS** and **synchronized blocks on individual bucket heads**. Reads are lock-free (`volatile Node[]`). Writes to an empty bucket use CAS (`compareAndSwapObject`). Writes to a non-empty bucket lock only that bucket head (synchronized on the first `Node`). This allows near-linear scalability — as many concurrent writers as there are buckets. The `compute/merge` operations also use this fine-grained locking.

[ADV] Q51. How does `Arrays.sort()` choose between TimSort and dual-pivot quicksort for Object arrays?

For `Object[]` (reference arrays), `Arrays.sort()` always uses TimSort (since Java 7) because: (1) objects need stable sort for correct semantics; (2) objects can implement `Comparable` in ways that make quicksort behave poorly; (3) the overhead of comparison via `Comparable/Comparator` is higher than primitive comparison, so TimSort's reduction in comparisons (via galloping) is more valuable. For primitive arrays (`int[]`, `long[]`, etc.), dual-pivot quicksort is used because: (1) stability is irrelevant for primitives; (2) in-place quicksort has better cache locality and avoids the $O(n)$ auxiliary memory that TimSort requires for merging.

[ADV] Q52. Explain the merge stack invariant in TimSort that prevents $O(n^2)$ merging behavior.

TimSort maintains a stack of pending run lengths. After pushing a new run, it checks two invariants: (1) $len[n-2] > len[n-1] + len[n]$; (2) $len[n-1] > len[n]$, where $len[i]$ is the length of the i -th run from the top. If either invariant is violated, TimSort merges the two smallest adjacent runs. These invariants ensure the run lengths grow at least as fast as the Fibonacci sequence, guaranteeing the total merge work is $O(n \log n)$. Without this invariant, degenerate cases (e.g., runs of lengths 1,1,1,1,...) would cause $O(n^2)$ merging as each tiny run merges one-by-one. The invariant forces balanced merges (similar to balanced merge sort), achieving the optimal $O(n \log n)$ bound.

Exercises — Part A: Coding Challenges

Exercise 1 — Generic Swap Method

Implement a generic swap method for any List using the Collections.swap pattern. Your method should work for any element type and validate bounds.

```
// Your implementation here:
public static <T> void swap(List<T> list, int i, int j) {
    // validate indices, then swap
    // Hint: Collections.swap(list, i, j) does:
    // T temp = list.set(i, list.get(j));
    // list.set(j, temp);
}
// Extend: implement rotateLeft(List, k) using swap
```

Exercise 2 — Top-N Frequency Elements

Write a method that takes a List and returns the top-3 most frequent elements in descending frequency order.

```
// Your implementation here:
public static <T> List<T> topN(List<T> items, int n) {
    // Step 1: Build frequency map using Collections.frequency or streams
    // Step 2: Sort entries by value (frequency) descending
    // Step 3: Extract top-n keys
    // Hint: Use Map.Entry.comparingByValue(Comparator.reverseOrder())
}
```

Exercise 3 — Multi-Field Stable Sort

Given Employee(name, department, salary), sort employees by department ascending, then salary descending, then name ascending. Use the Comparator API.

```
// Your implementation here:
employees.sort(
    Comparator.comparing(/* department */)
        .thenComparing(/* salary descending */)
        .thenComparing(/* name */)
);
// Verify stability: employees in same dept with same salary should stay in original relative order
```

Exercise 4 — Same Elements Regardless of Order

Write a method to check if two Lists contain the same elements regardless of order (treating duplicates correctly — [1,1,2] and [1,2] should return false).

```
// Your implementation here:
public static <T> boolean sameElements(List<T> a, List<T> b) {
    // Approach 1: sort and compare – O(n log n)
    // Approach 2: frequency map – O(n)
    // Hint for approach 2: build Map<T, Integer> for each list and compare maps
}
```

Exercise 5 — Bounded Rotating Buffer (ArrayDeque)

Implement a bounded rotating buffer that discards the oldest element when full. Use ArrayDeque. Support: add(T), List snapshot().

```
// Your implementation here:
public class RotatingBuffer<T> {
    private final int maxSize;
    private final ArrayDeque<T> deque;
    public RotatingBuffer(int maxSize) { /* ... */ }
```

```

public void add(T item) {
    // if deque.size() >= maxSize: remove oldest (pollFirst)
    // then addLast(item)
}

public List<T> snapshot() {
    return new ArrayList<>(deque);
}
}

```

Exercises — Part B: Debug & Fix

Bug 1 — ConcurrentModificationException (for-each removal)

```

// BUGGY CODE – find and fix the bug
List<String> names = new ArrayList<>(Arrays.asList("Alice", "Bob", "Charlie", "Bob"));
for (String name : names) {
    if (name.equals("Bob")) {
        names.remove(name); // BUG: throws ConcurrentModificationException
    }
}
// FIX: use removeIf(), iterator.remove(), or collect-then-removeAll

```

Bug 2 — Binary Search on Unsorted List

```

// BUGGY CODE – find and fix the bug
List<Integer> scores = new ArrayList<>(Arrays.asList(85, 42, 97, 13, 56));
int idx = Collections.binarySearch(scores, 97);
System.out.println("Found at: " + idx); // BUG: returns wrong result – list not sorted!
// FIX: Collections.sort(scores) BEFORE binarySearch, OR use indexOf(97) if sort is undesirable

```

Bug 3 — synchronizedList Iteration Without External Sync

```

// BUGGY CODE – find the thread-safety bug
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
// Thread 1: populating
syncList.add("a"); syncList.add("b"); syncList.add("c");
// Thread 2: iterating – BUG: not synchronized!
for (String s : syncList) {
    System.out.println(s); // may throw ConcurrentModificationException!
}
// FIX: synchronize on syncList during iteration:
// synchronized (syncList) { for (String s : syncList) { ... } }

```

Bug 4 — O(n) Membership Lookup Anti-Pattern

```

// BUGGY CODE – performance anti-pattern
List<String> allowedUsers = new ArrayList<>(loadUsersFromDB()); // 50,000 users
for (Request req : incoming100kRequests) {
    if (allowedUsers.contains(req.getUser())) { // O(n) * O(n) = O(n^2)!
        process(req);
    }
}
// FIX: convert to Set for O(1) lookup
Set<String> allowedUsersSet = new HashSet<>(allowedUsers); // O(n) once
for (Request req : incoming100kRequests) {
    if (allowedUsersSet.contains(req.getUser())) { // O(1) now
        process(req);
    }
}

```


Exercises — Part C: Multiple Choice Questions

MCQ 1: Which statement is true about ConcurrentHashMap's iterator?

- A) It throws ConcurrentModificationException if modified during iteration
- B) It iterates over a full snapshot taken at iterator creation
- C) It is weakly consistent — may or may not see concurrent changes
- D) It locks the entire map during iteration

Answer: C — ConcurrentHashMap's iterator is weakly consistent. It does not throw CME, does not snapshot the entire map, and does not lock.

MCQ 2: What does Arrays.sort(int[]) use internally?

- A) TimSort (stable, hybrid merge-insertion)
- B) Dual-pivot Quicksort (Yaroslavskiy, Bentley, Bloch)
- C) Single-pivot Quicksort with median-of-three
- D) Merge Sort

Answer: B — Arrays.sort(primitive[]) uses dual-pivot Quicksort. Arrays.sort(Object[]) uses TimSort.

MCQ 3: You call Collections.binarySearch(list, 99) on a sorted list [10,20,30,40] and get -5. What is the insertion point?

- A) 5
- B) -5
- C) 4
- D) 3

Answer: C — Insertion point = $-(\text{result}) - 1 = -(-5) - 1 = 4$. The value 99 would be inserted at index 4 (past the end).

MCQ 4: Which collection type is most appropriate for an LRU cache implementation?

- A) HashMap with manual eviction logic
- B) TreeMap ordered by access time
- C) LinkedHashMap in access-order mode with removeEldestEntry()
- D) ConcurrentHashMap with a PriorityQueue

Answer: C — LinkedHashMap(capacity, 0.75f, true) in access-order mode moves entries on access; removeEldestEntry() evicts the LRU entry.

MCQ 5: What is the time complexity of PriorityQueue.peek()?

- A) $O(\log n)$
- B) $O(n)$
- C) $O(1)$
- D) $O(n \log n)$

Answer: C — PriorityQueue is a min-heap. The minimum element is always at index 0 — peek() returns it in $O(1)$.

MCQ 6: List.of() vs Collections.unmodifiableList() — which statement is FALSE?

- A) List.of() does not allow null elements
- B) Collections.unmodifiableList() creates a view — changes to the backing list are visible
- C) List.of() creates a live view of the source collection
- D) Collections.unmodifiableList() throws UnsupportedOperationException on add()

Answer: C — List.of() creates a completely independent immutable list, NOT a view of any source. Option C is false.

Quick-Reference Cheat Sheet

Topic	Key Point
modCount / fail-fast	Increments on structural mod; iterator saves expectedModCount; CME if they diverge
Structural vs non-structural	add/remove/clear = structural; set() = non-structural (no modCount increment)
CopyOnWriteArrayList	Full array copy on write; iteration on snapshot; ideal for read-heavy, write-rare
ConcurrentHashMap iterator	Weak-consistency; captures Node[] ref at creation; may/may not see concurrent changes
Arrays.sort(primitive[])	Dual-pivot Quicksort; $O(n \log n)$ avg; NOT stable; in-place
Arrays.sort(Object[])	TimSort; $O(n \log n)$ worst, $O(n)$ best (sorted); stable; $O(n)$ aux memory
Collections.sort()	Delegates to list.sort() -> TimSort; stable; $O(n \log n)$
Collections.binarySearch()	Requires sorted list; returns -(insertion point)-1 if not found
TimSort runs	Exploits existing sorted order; $O(n)$ for sorted input; galloping mode for fast merges
Comparator.comparing()	Factory for key-extracting Comparator; thenComparing() chains decorators
List.of() / Set.of()	Java 9+; truly immutable; no nulls; specialized compact implementations
unmodifiableList() vs List.of()	unmodifiable = live view (backing changes visible); List.of() = independent immutable
HashMap worst case	$O(n)$ with collisions; Java 8+ treeifies buckets ≥ 8 nodes -> $O(\log n)$ worst
LinkedHashMap LRU cache	new LinkedHashMap(cap, 0.75f, true) + override removeEldestEntry()
EnumMap vs HashMap	EnumMap: array-backed, $O(1)$, ordered by declaration, no null keys, faster
removeIf() efficiency	ArrayList.removeIf() uses bitmask approach: $O(n)$ total vs $O(n^2)$ for iterator removal
synchronizedList iteration	MUST externally sync: synchronized(list) { for(...) {} } or use CopyOnWriteArrayList
PriorityQueue	Min-heap; peek $O(1)$; offer/poll $O(\log n)$; contains $O(n)$; use PriorityBlockingQueue for concurrency

Next: Sub-Chapter 1.7 — Generics

Type Parameters · Wildcards · Bounded Types · Type Erasure · Heap Pollution · Generic Methods

Java Backend Architecture Interview Series · Sub-Chapter 1.6 of 12 · Senior/Principal Engineer Level